

MANUAL DEL PROGRAMADOR

FinCredit — Credit & Loan Management System

Proyecto Final POO | Autores: Cristian Castro y Henry Garrido | Versión: 1.0 | Fecha: 2026

TABLA DE CONTENIDO

1. Descripción general del sistema
 2. Arquitectura y patrones de diseño
 3. Diagrama de paquetes
 4. Descripción de cada clase
 5. Pilares de POO aplicados
 6. Flujo de datos principal
 7. Guía para extender el sistema
 8. Convenciones de código
-

1. DESCRIPCIÓN GENERAL DEL SISTEMA

FinCredit es una aplicación de escritorio desarrollada en Java con interfaz gráfica Swing. Permite registrar clientes, solicitar préstamos, evaluar crediticios y generar tablas de amortización. El sistema está diseñado como proyecto académico para demostrar los cuatro pilares de la programación orientada a objetos: herencia, polimorfismo, encapsulamiento y abstracción.

Tecnologías utilizadas:

Tecnología	Uso
Java 11+	Lenguaje de programación
Java Swing	Interfaz gráfica de usuario
<code>java.time.LocalDateTime</code>	Timestamps de creación
<code>java.util.LinkedHashMap</code>	Almacenamiento ordenado de entidades
Patrón Singleton	<code>LoanRegistry</code> y <code>LoanProductFactory</code>
Patrón Factory	<code>LoanProductFactory</code> para crear productos
Patrón Strategy	<code>IEvaluable</code> / <code>CreditEvaluator</code>
CardLayout (Swing)	Navegación entre paneles

2. ARQUITECTURA Y PATRONES DE DISEÑO

El sistema sigue una arquitectura en capas:

CAPA DE PRESENTACIÓN
gui/ → MainFrame, DashboardPanel, ClientsPanel, LoansPanel, NewClientPanel, NewLoanPanel
CAPA DE LÓGICA
logic/ → CreditEvaluator LoanProductFactory
CAPA DE MODELO
model/ → Person, Client, LoanOfficer, Loan, LoanProduct, AmortizationRow, EvaluationResult, ConsumerLoan, EducationalLoan, MortgageLoan, FreeInvestmentLoan
CAPA DE DATOS / REPOSITORIO
registry/ → LoanRegistry
CONTRATOS (API)
interfaces/ → IEvaluable, ILoanable

Patrones aplicados

Singleton — `LoanRegistry` y `LoanProductFactory` tienen una única instancia en toda la aplicación.

java

```
// Uso desde cualquier clase:
LoanRegistry registry = LoanRegistry.getInstance();
LoanProductFactory factory = LoanProductFactory.getInstance();
```

Factory — `LoanProductFactory` centraliza la creación y entrega de los 4 tipos de producto.

java

```
LoanProduct product = factory.getProduct("MORTGAGE"); // retorna MortgageLoan
```

Strategy — La evaluación crediticia es intercambiable vía `IEvaluable`. Se puede inyectar un evaluador diferente sin modificar el resto del sistema.

```
java
```

```
registry.setEvaluator(new CreditEvaluator(0.25)); // umbral personalizado
```

3. DIAGRAMA DE PAQUETES

```
com.fincredit
|
├── interfaces
|   ├── ILoanable          ← contrato para productos de préstamo
|   └── IEvaluable         ← contrato para evaluadores de crédito
|
├── model
|   ├── Person (abstract) ← superclase de Client y LoanOfficer
|   |   ├── Client        ← extiende Person, agrega finanzas
|   |   └── LoanOfficer    ← extiende Person, agrega sucursal
|   |
|   ├── LoanProduct (abstract, implements ILoanable)
|   |   ├── MortgageLoan   ← 7.5% / 360 meses
|   |   ├── EducationalLoan ← 5.0% / 120 meses
|   |   ├── ConsumerLoan   ← 12.0% / 60 meses
|   |   └── FreeInvestmentLoan ← 14.0% / 48 meses
|   |
|   ├── Loan               ← préstamo (tiene LoanProduct, Client, amortization)
|   ├── AmortizationRow    ← una fila de la tabla de amortización
|   └── EvaluationResult   ← resultado de la evaluación crediticia
|
├── logic
|   ├── CreditEvaluator    ← implements IEvaluable, aplica regla del 30%
|   └── LoanProductFactory ← Singleton + Factory de LoanProduct
|
├── registry
|   └── LoanRegistry       ← Singleton, repositorio central de datos
|
└── gui
    ├── Main              ← punto de entrada, lanza MainFrame
    ├── MainFrame         ← JFrame principal con sidebar y CardLayout
    ├── DashboardPanel    ← estadísticas y catálogo de productos
    ├── ClientsPanel      ← tabla de clientes registrados
    └── LoansPanel        ← tabla de todos los préstamos
```

- └─ NewClientPanel ← formulario de registro de cliente
- └─ NewLoanPanel ← formulario de solicitud de préstamo

4. DESCRIPCIÓN DE CADA CLASE

Person — `model/Person.java`

Clase abstracta que actúa como superclase de toda persona en el sistema. Define los atributos comunes (`id`, `name`, `document`, `email`, `createdAt`) y declara dos métodos abstractos que cada subclase debe implementar.

Atributos principales:

Atributo	Tipo	Modificador	Descripción
<code>id</code>	String	<code>protected final</code>	Identificador único, no modificable
<code>name</code>	String	<code>protected</code>	Nombre completo
<code>document</code>	String	<code>protected</code>	Número de documento (CC, NIT, etc.)
<code>email</code>	String	<code>protected</code>	Correo electrónico
<code>createdAt</code>	LocalDateTime	<code>protected final</code>	Fecha y hora de creación

Métodos abstractos: `getRole()`, `getSummary()`

Client — `model/Client.java`

Extiende `Person`. Representa un cliente financiero con ingresos y gastos mensuales. Implementa los métodos de evaluación financiera usados por `CreditEvaluator`.

Atributos propios:

Atributo	Tipo	Descripción
<code>monthlyIncome</code>	double	Ingreso mensual bruto
<code>monthlyExpenses</code>	double	Gastos mensuales fijos
<code>loanIds</code>	List<String>	IDs de los préstamos asociados
<code>PAYMENT_CAPACITY_RATIO</code>	static final double	Constante 0.30 (30%)

Métodos financieros clave:

Método	Fórmula / Lógica	Retorno
<code>getNetIncome()</code>	<code>monthlyIncome - monthlyExpenses</code>	double
<code>getMaxPaymentCapacity()</code>	<code>monthlyIncome * 0.30</code>	double
<code>canAfford(pay)</code>	<code>pay <= getMaxPaymentCapacity()</code>	boolean
<code>getCreditScore()</code>	Ratio net/income → AAA/AA/A/BBB/C	String

Tabla de credit score:

Ratio <code>getNetIncome() / monthlyIncome</code>	Calificación
≥ 0.50	AAA (excelente)
≥ 0.35	AA (muy bueno)
≥ 0.20	A (bueno)
≥ 0.05	BBB (regular)
< 0.05	C (deficiente)

`LoanOfficer` — `model/LoanOfficer.java`

Extiende `Person`. Modela a un oficial de crédito con sucursal asignada y contadores de decisiones. En la versión actual no está expuesto en la GUI, pero está disponible para expansiones futuras.

`LoanProduct` — `model/LoanProduct.java`

Clase abstracta que implementa `ILoanable`. Contiene la lógica central de cálculo financiero compartida por los 4 tipos de producto.

Fórmula de cuota mensual (líneas 26-32):

```
i = annualRate / 100 / 12           (tasa mensual)
factor = (1 + i) ^ terms
cuota = principal * (i * factor) / (factor - 1)
```

Si la tasa es 0% (préstamo sin interés): `cuota = principal / terms`

Generación de tabla de amortización (líneas 35-52): Para cada período se calcula:

- `interest = balance * i`
- `capitalPayment = payment - interest`
- `balance -= capitalPayment`

Métodos abstractos que cada subclase implementa:

Método	<code>MortgageLoan</code>	<code>EducationalLoan</code>	<code>ConsumerLoan</code>	<code>FreeInvestmentLoan</code>
<code>getBaseRate()</code>	7.5%	5.0%	12.0%	14.0%
<code>getMaxTerm()</code>	360 meses	120 meses	60 meses	48 meses
<code>getDescription()</code>	"Long-term real-estate..."	"Student and academic..."	"General consumer..."	"Flexible-purpose..."

`Loan` — `model/Loan.java`

Representa un préstamo creado en el sistema. Es inmutable en sus campos financieros (todos `final` excepto `status` y `evaluationResult`). Al construirse, calcula automáticamente la cuota mensual y genera la tabla de amortización completa.

Estados posibles (enum interno): `PENDING`, `APPROVED`, `REJECTED`

Cálculos disponibles:

Método	Descripción
<code>getMonthlyPayment()</code>	Cuota mensual calculada en constructor
<code>getTotalCost()</code>	<code>monthlyPayment * terms</code>
<code>getTotalInterest()</code>	<code>getTotalCost() - principal</code>
<code>isApproved()</code>	<code>status == APPROVED</code>

AmortizationRow — `model/AmortizationRow.java`

Clase inmutable (todos los campos `final`). Representa una fila en la tabla de amortización: período, cuota total, intereses, abono a capital y saldo restante.

EvaluationResult — `model/EvaluationResult.java`

Clase inmutable que encapsula el resultado de la evaluación crediticia: si fue aprobado, la cuota, la capacidad máxima, el ratio pago/ingreso, el motivo en texto y el nombre del evaluador.

CreditEvaluator — `logic/CreditEvaluator.java`

Implementa `IEvaluable`. Aplica la regla de negocio principal: un préstamo se aprueba si la cuota mensual no supera el 30% del ingreso mensual del cliente.

Lógica de evaluación (líneas 27-43):

```
java
maxCapacity = client.getMaxPaymentCapacity(); // ingreso * 0.30
approved    = monthlyPayment <= maxCapacity;
```

El umbral del 30% está definido como constante `DEFAULT_THRESHOLD = 0.30` y puede ser modificado mediante el constructor con parámetro o inyectado vía `registry.setEvaluator(...)`.

LoanProductFactory — `logic/LoanProductFactory.java`

Patrón Singleton + Factory. Mantiene un catálogo `LinkedHashMap<String, LoanProduct>` con los 4

productos registrados al inicializar. El orden de inserción se preserva (MORTGAGE → EDUCATIONAL → CONSUMER → FREE_INVESTMENT), lo cual es relevante para la GUI ya que los combos muestran los productos en este orden.

`LoanRegistry` — `registry/LoanRegistry.java`

Patrón Singleton. Es el corazón del sistema: centraliza todos los clientes y préstamos, y orquesta la creación de préstamos con su evaluación.

Flujo de `requestLoan()`:

- 1. Recupera el `Client` por `clientId`.
- 2. Recupera el `LoanProduct` por `productType` desde la fábrica.
- 3. Determina la tasa (custom o base del producto).
- 4. Crea el objeto `Loan` (que internamente calcula cuota y tabla de amortización).
- 5. Llama a `evaluator.evaluate(client, loan.getMonthlyPayment())`.
- 6. Según el resultado, llama a `loan.approve(result)` o `loan.reject(result)`.
- 7. Guarda el préstamo en el mapa y registra el ID en el cliente.

Datos precargados por `seedData()` al iniciar:

Cliente	Documento	Ingreso	Gastos	Préstamo solicitado
Ana Martínez	CC-1001	\$5,000,000	\$1,500,000	MORTGAGE \$180M / 240 meses
Carlos Rivera	CC-1002	\$8,000,000	\$3,200,000	EDUCATIONAL \$25M / 60 meses
Laura Torres	CC-1003	\$3,200,000	\$2,600,000	CONSUMER \$15M / 36 meses
Diego Suárez	CC-1004	\$12,000,000	\$4,000,000	FREE_INVESTMENT \$50M / 48 meses

5. PILARES DE POO APLICADOS

Herencia

Person (abstracta)

- └ Client → agrega monthlyIncome, loanIds, credit score
- └ LoanOfficer → agrega branch, contadores de decisiones

LoanProduct (abstracta, implements ILoanable)

— MortgageLoan	→ tasa 7.5%, max 360 meses
— EducationalLoan	→ tasa 5.0%, max 120 meses
— ConsumerLoan	→ tasa 12.0%, max 60 meses
— FreeInvestmentLoan	→ tasa 14.0%, max 48 meses

Los atributos comunes (`id`, `name`, `document`, `email`) se definen una sola vez en `Person` y son heredados por todas las subclases, evitando duplicación de código.

Polimorfismo

Gracias al polimorfismo, la GUI puede iterar sobre una lista de `LoanProduct` genéricos y cada objeto se comporta según su tipo real:

java

```
// DashboardPanel.java línea 216 – cada p es un subtipo distinto
for (LoanProduct p : LoanProductFactory.getInstance().getAllProducts()) {
    p.getDescription(); // MortgageLoan.getDescription(), ConsumerLoan.getDescription()
    p.getBaseRate();     // 7.5, 5.0, 12.0, 14.0 según el objeto real
    p.getMaxTerm();      // 360, 120, 60, 48 según el objeto real
}
```

Encapsulamiento

Todos los atributos de las clases del modelo son `private` o `protected`, y el acceso externo se realiza únicamente a través de getters y setters. Por ejemplo:

- `loanIds` en `Client` se expone como lista inmutable (`Collections.unmodifiableList`) para que nadie pueda modificarla externamente.
- Los campos de `Loan` son todos `final` (excepto `status`), garantizando inmutabilidad de los datos financieros.

Abstracción

- `Person` es abstracta: no tiene sentido crear un objeto "Person" genérico; siempre será un `Client` o un `LoanOfficer`.
- `LoanProduct` es abstracta: no tiene sentido crear un producto sin tipo, tasa ni plazo definidos.
- `IEvaluable` e `ILoanable` son contratos (interfaces) que separan el qué hace el sistema del cómo lo hace.

6. FLUJO DE DATOS PRINCIPAL

Registro de cliente

```
NewClientPanel.registerClient()
  → valida campos del formulario
  → LoanRegistry.registerClient(name, doc, email, income, expenses)
    → crea new Client(id, name, doc, email, income, expenses)
    → guarda en clients map
  → retorna Client con ID generado ("C0001", "C0002", ...)
```

Solicitud de préstamo

```
NewLoanPanel.submitLoan()
  → valida campos del formulario
  → LoanRegistry.requestLoan(clientId, productType, principal, terms, customRate)
    → recupera Client del mapa
    → recupera LoanProduct del catálogo (Factory)
    → crea new Loan(id, clientId, product, principal, terms, rate)
      → LoanProduct.calculateMonthlyPayment(principal, rate, terms)
      → LoanProduct.generateAmortizationTable(principal, rate, terms)
    → CreditEvaluator.evaluate(client, monthlyPayment)
      → si cuota <= 30% del ingreso → APPROVED
      → si cuota > 30% del ingreso → REJECTED
    → loan.approve(result) o loan.reject(result)
    → guarda en loans map
    → client.addLoanId(loanId)
  → retorna Loan con resultado
```

7. GUÍA PARA EXTENDER EL SISTEMA

Agregar un nuevo tipo de préstamo

1. Crear nueva clase en `model/` que extienda `LoanProduct`:

```
java
```

```
public class VehicleLoan extends LoanProduct {
    public VehicleLoan() { super("VEHICLE", "🚗"); }

    @Override public double getBaseRate() { return 10.5; }
    @Override public int getMaxTerm() { return 72; }
    @Override public String getDescription() { return "Vehicle financing"; }
}
```

2. Registrarla en `LoanProductFactory` (constructor, línea ~21):

```
java
register(new VehicleLoan());
```

Eso es todo. La GUI detectará el nuevo producto automáticamente en los combos y el catálogo del Dashboard, gracias al polimorfismo.

Agregar un nuevo evaluador de crédito

1. Crear clase en `logic/` que implemente `IEvaluable`.
2. Inyectarlo en el registry:

```
java
registry.setEvaluator(new MiNuevoEvaluador());
```

Agregar persistencia (base de datos)

Actualmente los datos viven en memoria. Para agregar persistencia:

1. Modificar `LoanRegistry.seedData()` para cargar datos desde archivo o BD.
2. Agregar métodos `save()` que serialicen los mapas.
3. Llamar a `save()` cuando se registre un cliente o préstamo.

8. CONVENCIONES DE CÓDIGO

Convención

Ejemplo

Clases en PascalCase

`LoanProduct`, `CreditEvaluator`

Convención	Ejemplo
Métodos y variables en camelCase	<code>getMonthlyPayment()</code> , <code>clientSeq</code>
Constantes en UPPER_SNAKE_CASE	<code>PAYMENT_CAPACITY_RATIO</code> , <code>DEFAULT_THRESHOLD</code>
Paquetes en lowercase	<code>com.fincredit.model</code>
IDs de clientes	Formato <code>C0001</code> , <code>C0002</code> , ...
IDs de préstamos	Formato <code>L0001</code> , <code>L0002</code> , ...
Colores UI	Definidos como constantes <code>static final Color</code> en cada panel
Getters sin prefijo para interfaces	<code>getType()</code> , <code>getIcon()</code> (no <code>isType()</code>)